A Project Report On

Flight Delay Prediction Using Machine Learning

Submitted in partial fulfillment of the requirement for the award of the degree

Master of Computer Applications(MCA)
from
Marwadi University

Academic Year 2026 – 27

Kashyap Bharad (92500584001)
Chirag Pandya (92500584070)

Internal Guide
DR. DIVYAKANT MEVA

Rajkot-Morbi Road, At &PO :Gauridad, Rajkot 360 003. Gujarat. India.

Marwadi University
Marwadi Chandarana Group

NAAC A+

Faculty of Computer Applications (FoCA)

Certificate

This is to certify that the project work entitled

Flight Delay Prediction Using Machine Learning

submitted in partial fulfillment of the requirement for the award of the degree of

Master of Computer Applications (MCA)

of the

Marwadi University

is a result of the bonafide work carried out by

Kashyap Bharad (92500584001)
Chirag Pandya (92500584070)

during the academic year 2026-27

Faculty Guide	HOD	Dean

DECLARATION

We hereby declare that this project work entitled **Flight Delay Prediction Using Machine Learning** is a record done by us.

We also declare that the matter embodied in this project is genuine work done by us and has not been submitted whether to this University or to any other University/Institute for the fulfillment of the requirement of any course of study.

Place: Rajkot

Date:

Kashyap Bharad (92500584001) Signature:_____
Chirag Pandya (92500584070) Signature:_____

ACKNOWLEDGEMENT

It is indeed a great pleasure to express our thanks and gratitude to all those who helped us. No serious and lasting achievement or success one can ever achieve without the help of friendly guidance and co-operation of so many people involved in the work.

We are very thankful to our guide **DR. DIVYAKANT MEVA,** the person who makes us to follow the right steps during our project work. We express our deep sense of gratitude to for his guidance, suggestions and expertise at every stage. A part from that his valuable and expertise suggestion during documentation of our report indeed help us a lot.

Thanks to our friend and colleague who have been a source of inspiration and motivation that helped to us during our project work.

We are heartily thankful to the Dean of our department **Dr. Rijwan Khan** for giving us an opportunity to work over this project and for their end-less and great support to all other people who directly or indirectly supported and help us to fulfil our task.

Kashyap Bharad (92500584001) Signature:_____
Chirag Pandya (92500584070) Signature:_____

CONTENTS

Figure Index

Table Index

1.Introduction

- Flight delays are a common problem in the aviation industry and can affect passengers, airlines, airports, and overall flight schedules. Delays may occur due to different factors such as weather conditions, airline operations, aircraft type, travel distance, and other environmental conditions. Predicting flight delays in advance can help in better planning and decision-making.
- The main objective of this project is to develop a Machine Learning-based system that predicts whether a flight will be delayed or not. The dataset used in this project contains flight-related and environmental information, including airline, origin, destination, weather conditions, aircraft type, temperature, wind speed, distance, and holiday information.
- Before applying Machine Learning algorithms, the dataset is processed using different data preprocessing techniques. Missing numerical values are handled using the median, while missing categorical values are handled using the mode. Categorical features are converted into numerical values using Label Encoding. Outliers are identified and removed, unnecessary and leakage-related features are removed, and feature scaling is performed using StandardScaler.
- The main steps performed in this project are:

    - Data cleaning and preprocessing
    - Missing value handling
    - Outlier detection and removal
    - Categorical data encoding
    - Removal of unnecessary and leakage-related features
    - Train-test splitting using 70-30 and 80-20 ratios
    - Feature scaling using StandardScaler

- Six Machine Learning classification algorithms are used and compared in this project: Logistic Regression, K-Nearest Neighbors (KNN), Decision Tree, Naive Bayes, Random Forest, and Linear Support Vector Machine (SVM).
- The performance of the models is evaluated using Accuracy, Precision, Recall, F1-Score, and Confusion Matrix. In addition, 5-Fold Cross Validation is used to evaluate the consistency of the models, while Grid Search is used for hyperparameter tuning.
- Finally, the performance of all models is compared, and the best-performing model is selected for the Flight Delay Prediction task. This project demonstrates the use of Machine Learning techniques to analyze flight-related and environmental data and predict possible flight delays in advance.

2.Dataset Description

2.1 Overview

Property	Details
Dataset Name	Flight Delay Prediction Dataset
File	Flight_Delay_Prediction_Dataset_Raw.csv
Total Records	Approximately 63,729 flight records
Data Type	Flight-related and environmental data
Target Variable	Delay_Status
Task Type	Supervised Binary Classification

Table 1: Dataset Overview

The dataset contains flight-related and environmental information that can influence whether a flight is delayed. The objective of this project is to use the available features to predict the flight delay status. Since the target variable has two possible outcomes, delayed or not delayed, the problem is treated as a binary classification problem.

2.2 Feature Matrix (All Columns)

#	Column Name	Description	Type	Unit / Values
1	Flight_ID	Unique identifier for each flight	Identifier	—
2	Date	Date associated with the flight	Date	Date
3	Airline	Airline operating the flight	Categorical	Airline Name
4	Flight_Number	Flight number of the flight	Identifier	—
5	Origin	Location from which the flight departs	Categorical	Airport / Location
6	Destination	Location where the flight arrives	Categorical	Airport / Location
7	Weather	Weather condition associated with the flight	Categorical	Weather Category
8	Aircraft_Type	Type of aircraft used for the flight	Categorical	Aircraft Category
9	Temperature_C	Temperature during flight conditions	Numerical	°C
10	Wind_Speed_kmph	Wind speed during flight conditions	Numerical	km/h
11	Distance_km	Total flight travel distance	Numerical	km
12	Holiday	Indicates whether the date is a holiday	Categorical	Yes / No
13	Departure_Delay_Min	Departure delay duration	Numerical	Minutes
14	Delay_Status	Indicates whether the flight is delayed	Target	0 / 1

Table 2: Feature Matrix — All 17 Columns with Description, Type & Unit

2.3 Target Variable

Value	Meaning
0	Flight is not delayed
1	Flight is delayed

Table 3: Target Variable Description

The target variable **Delay_Status** represents the outcome that the Machine Learning models predict. The value 0 represents a flight without a delay, while the value 1 represents a delayed flight.

2.4 Dataset Characteristics

- Contains both numerical and categorical features.
- Includes flight-related and environmental information.
- Contains identifier and date-related columns.
- Uses **Delay_Status** as the target variable.
- Represents a supervised binary classification problem.
- Requires preprocessing before Machine Learning models are applied.

3. Data Preprocessing & Feature Engineering

3.1 Steps Performed

1. **Data Loading:** The Flight Delay Prediction dataset was loaded into the project using the pandas library.
2. **Initial Data Inspection:** The dataset structure, column names, data types, and missing values were checked before preprocessing.
3. **Missing Value Handling:** Missing numerical values were handled using the **median**, while missing categorical values were handled using the **mode**.
4. **Categorical Feature Encoding:** Categorical features such as **Airline, Origin, Destination, Weather, Aircraft_Type, and Holiday** were converted into numerical values using **Label Encoding**.
5. **Removal of Unnecessary Features:** Identifier and unnecessary features such as **Flight_ID, Date, and Flight_Number** were removed because they do not directly contribute to the prediction.
6. **Removal of Data Leakage Feature: Departure_Delay_Min** was removed to prevent data leakage because it can directly reveal information related to the flight delay status.
7. **Outlier Detection and Removal:** Outliers in numerical features were identified using the **Interquartile Range (IQR) method** and removed from the dataset.
8. **Feature and Target Separation:** The **Delay_Status** column was selected as the target variable, while the remaining processed columns were used as input features.
9. **Class Distribution Analysis:** The distribution of delayed and non-delayed flights was checked to understand the balance between the two target classes.

3.2 Train-Test Split

The processed dataset was divided into training and testing data using two different train-test split ratios: **70-30 and 80-20**. Random state was kept fixed to ensure reproducible results.

Split	Training Data	Testing Data
70-30 Split	70%	30%
80-20 Split	80%	20%

Table 4: Train-Test Split Ratios

The two different train-test splits were used to compare the performance of Machine Learning models under different training and testing conditions. The **80-20 split** was later used for the final model comparison and hyperparameter tuning.

3.3 Feature Scaling

Feature scaling was performed using **StandardScaler**. It standardizes the feature values so that features with larger numerical ranges do not have an unnecessary influence on the Machine Learning models..

Model	Feature Scaling	Reason
Logistic Regression	StandardScaler	Improves convergence and provides features on a similar scale
K-Nearest Neighbors (KNN)	StandardScaler	Distance-based algorithm and sensitive to feature scale
Decision Tree	StandardScaler	Not required, but applied for consistent preprocessing
Naive Bayes	StandardScaler	Applied for consistent preprocessing
Random Forest	StandardScaler	Not required, but applied for consistent preprocessing
Linear Support Vector Machine (SVM)	StandardScaler	Sensitive to feature scale and improves model convergence

Table 5: Feature Scaling Strategy for Machine Learning Models

StandardScaler transforms the numerical features so that they have a mean close to **0** and a standard deviation close to **1**. The scaler was fitted only on the training data, and the same transformation was then applied to the testing data. This prevents information from the testing data from influencing the training process.

4. Machine Learning Models Used

Six classification algorithms were selected for the Flight Delay Prediction project. These models were chosen because they use different approaches to classification, allowing us to compare their performance and identify the most suitable model for predicting whether a flight will be delayed or not.

The models used are:

1. Logistic Regression
2. K-Nearest Neighbors (KNN)
3. Decision Tree
4. Naive Bayes
5. Random Forest
6. Linear Support Vector Machine (SVM)

4.1 Logistic Regression

Algorithm: Logistic Regression — sklearn.linear_model.LogisticRegression

How it works:
Logistic Regression is a classification algorithm that estimates the probability of an observation belonging to a particular class. In this project, it predicts whether a flight is **Delayed (1)** or **Not Delayed (0)**.

The model uses the input flight and environmental features to calculate the probability of the flight being delayed. A suitable classification threshold is then used to assign the final class.

Hyperparameter:

- max_iter = 10000

Purpose:
Logistic Regression is used as one of the basic classification models and provides a useful baseline for comparing the performance of the other algorithms.

Evaluation:
The model was trained using both **70-30 and 80-20 train-test splits**, and its performance was evaluated using accuracy, precision, recall, F1-score, and confusion matrix.

4.2 K-Nearest Neighbors (KNN)

Algorithm: K-Nearest Neighbors — sklearn.neighbors.KNeighborsClassifier

How it works:
KNN classifies a new data point based on the classes of its nearest neighboring data points. The model calculates the distance between observations and uses the majority class among the nearest neighbors for prediction.

Hyperparameter:

- n_neighbors = 5

Purpose:
KNN was included to evaluate a distance-based classification approach. Since the distance between observations is important for KNN, the input features were scaled before applying the algorithm.

Evaluation:
KNN was tested using both **70-30 and 80-20 train-test splits** and evaluated using accuracy, precision, recall, and F1-score.

4.3 Decision Tree

Algorithm: Decision Tree Classifier — sklearn.tree.DecisionTreeClassifier

How it works:
A Decision Tree makes predictions by dividing the dataset into smaller groups using a series of decision rules. Each split is based on the input features, and the final leaf node determines whether the flight is delayed or not.

Hyperparameter:

- random_state = 42

Purpose:
The Decision Tree was selected because it can capture non-linear relationships between flight-related and environmental features. It is also relatively easy to understand and interpret.

Evaluation:
The model was trained using both **70-30 and 80-20 splits** and evaluated using classification metrics and a confusion matrix.

4.4 Naive Bayes

Algorithm: Gaussian Naive Bayes — sklearn.naive_bayes.GaussianNB

How it works:
Naive Bayes calculates the probability of each class based on the input features. It assumes that the features are conditionally independent given the class.

For this project, Gaussian Naive Bayes was used because it is suitable for numerical features and provides a simple probabilistic classification approach.

Hyperparameters:

- Default Gaussian Naive Bayes parameters were used.

Purpose:
Naive Bayes provides a fast and simple classification method that can be compared with more complex algorithms.

Evaluation:
The model was evaluated using both train-test splits using accuracy, precision, recall, and F1-score.

4.5 Random Forest

Algorithm: Random Forest Classifier — sklearn.ensemble.RandomForestClassifier

How it works:
Random Forest is an ensemble learning algorithm that combines multiple Decision Trees. Each tree is trained using a random sample of the training data and a random selection of features. The final prediction is based on the combined predictions of the trees.

Hyperparameters:

- n_estimators = 100
- random_state = 42

Purpose:
Random Forest was selected because it can handle non-linear relationships and interactions between multiple features. It also generally provides good performance on classification problems.

Model-Specific Analysis:
Random Forest was further analyzed using:

- Confusion Matrix
- Classification Report
- 5-Fold Cross-Validation
- Grid Search for hyperparameter tuning
- Comparison with the other classification algorithms

Random Forest achieved the best overall performance among the initial models and was selected as the final model for the project.

4.6 Linear Support Vector Machine (SVM)

Algorithm: Linear Support Vector Machine — sklearn.svm.LinearSVC

How it works:
SVM finds a decision boundary that separates the different classes. In this project, a **linear SVM** was used to separate delayed and non-delayed flights based on their feature values.

The linear kernel was selected instead of the original SVC(kernel="linear") implementation because LinearSVC is considerably faster for a large dataset.

Hyperparameter:

- C was tuned using Grid Search.
- class_weight = "balanced" was used to give more importance to the minority class.

Purpose:
SVM was included as a strong classification method that can work effectively with scaled numerical features.

Evaluation:
The model was evaluated using accuracy, precision, recall, F1-score, 5-fold cross-validation, and Grid Search.

Model Comparison

The six algorithms provide different approaches to the same classification problem:

Model	Approach	Main Purpose
Logistic Regression	Linear classification	Baseline model
KNN	Distance-based	Neighbor-based classification
Decision Tree	Tree-based	Non-linear decision rules
Naive Bayes	Probabilistic	Fast probabilistic classification
Random Forest	Ensemble	High-performance classification
Linear SVM	Margin-based	Linear class separation

Table 6: Summary of the Machine Learning Models Used for Flight Delay Prediction and Their Primary Approaches.

5. Evaluation Metrics

5.1 Evaluation Metrics Used

All six classification models were evaluated on the held-out test data. Different evaluation metrics were used to understand not only the overall accuracy of the models but also how well they identify **Delayed** and **Not Delayed** flights.

Metric	Formula / Basis	Interpretation
Accuracy	(TP + TN) / (TP + TN + FP +FN)	Percentage of total predictions that are correct.
Precision	TP / (TP + FP)	Measures how many flights predicted as delayed were actually delayed.
Recall	TP / (TP + FN)	Measures how many of the actually delayed flights were correctly identified.
F1-Score	2 × (Precision × Recall) / (Precision + Recall)	Harmonic mean of precision and recall; useful when both are important.
Confusion Matrix	TP, TN, FP, FN	Shows the number of correct and incorrect predictions for each class.
Cross-Validation Accuracy	Mean accuracy across 5 folds	Measures how consistently the model performs across different subsets of the training data.

Table 7: Evaluation Metrics Used for Flight Delay Prediction

5.2 Classification Report

The **classification report** was used to obtain precision, recall, and F1-score separately for both classes:

- **Class 0 — No Delay**
- **Class 1 — Delay**

This provides a more detailed evaluation than accuracy alone, especially because the dataset contains more delayed flights than non-delayed flights.

5.3Confusion Matrix

The confusion matrix was used for the final Random Forest model to visually and numerically examine the correct and incorrect predictions. It contains:

- **True Negative (TN):** No-delay flight correctly predicted as no delay.
- **True Positive (TP):** Delayed flight correctly predicted as delayed.
- **False Positive (FP):** No-delay flight incorrectly predicted as delayed.
- **False Negative (FN):** Delayed flight incorrectly predicted as no delay.

5.4 5-Fold Cross-Validation

- 5-Fold Cross-Validation was used to check the consistency and reliability of the six classification models. The training dataset was divided into five subsets, where four subsets were used for training and the remaining subset was used for validation. This process was repeated five times so that each subset was used once for validation.

- The mean accuracy and standard deviation were calculated for each model. A higher mean accuracy indicates better overall performance, while a lower standard deviation indicates more consistent performance across different folds.

Model	Mean Accuracy	Standard Deviation
Logistic Regression	73.15%	0.24%
KNN	76.70%	0.42%
Decision Tree	79.60%	0.45%
Naive Bayes	75.99%	0.24%
Random Forest	**84.71%**	0.34%
Linear SVM	66.28%	0.49%

Table 8: 5-Fold Cross-Validation Results

5.5 Grid Search and Hyperparameter Tuning

Grid Search was applied to all six Machine Learning models to find suitable hyperparameter values. GridSearchCV with 5-fold cross-validation was used for this process. Different parameter combinations were tested, and the combination providing the highest cross-validation accuracy was selected as the best configuration.

Model	Best Parameters	Best CV Accuracy
Logistic Regression	C = 0.1	73.16%
KNN	n_neighbors = 7	77.23%
Decision Tree	max_depth = 10	84.51%
Naive Bayes	var_smoothing = 1e-9	75.99%
Random Forest	n_estimators = 100, max_depth = 20	**84.82%**
Linear SVM	C = 0.1	66.28%

Table 9: Best Hyperparameters and Cross-Validation Accuracy

5.6 Model Accuracy Comparison

The accuracy of all six classification models was compared using both 70–30 and 80–20 train-test splits.

Model	70–30 Accuracy	80–20 Accuracy
Logistic Regression	73.64%	73.58%
KNN	76.81%	77.01%
Decision Tree	80.26%	79.57%
Naive Bayes	76.11%	76.17%
Random Forest	**84.92%**	**84.95%**
Linear SVM	—	65.80%

Table 10: Model Accuracy Comparison

5.7 Tuned Model Accuracy Comparison

After Grid Search, the best hyperparameters obtained for each model were used to train the final tuned models. Each tuned model was then evaluated on the **80–20 test dataset**. The tuned accuracies were compared to identify the best-performing algorithm for the Flight Delay Prediction task.

Model	Tuned Accuracy
Random Forest	**84.76%**
Decision Tree	**84.47%**
KNN	77.47%
Naive Bayes	76.17%
Logistic Regression	73.60%
Linear SVM	65.79%

Table 11: Tuned Model Accuracy Comparison

5.8 Best Model Selection

Based on the tuned model results, **Random Forest** was selected as the final model for the Flight Delay Prediction system. It achieved the highest test accuracy among all six tuned classification models.

- **Best Model:** Random Forest
- **Train-Test Split:** 80–20
- **Best CV Accuracy:** 84.82%
- **Final Test Accuracy: 84.76%**

The final Random Forest model achieved a precision, recall, and F1-score of **0.89** for the **Delay** class. This indicates that the model provides strong performance in identifying delayed flights.

Final Classification Report:

Class	Precision	Recall	F1-Score
No Delay (0)	0.76	0.75	0.75
Delay (1)	0.89	0.89	0.89
Overall Accuracy			**84.76%**

Table 12: Final Random Forest Classification Results

6. Technology Stack

Component	Technology	Purpose / Usage
Programming Language	**Python**	Development of the complete Machine Learning project
Data Handling	**pandas**	Loading, cleaning and manipulating the dataset
Data Processing	**pandas**	Handling and processing numerical data
Missing Value Handling	**SimpleImputer**	Filling missing numerical and categorical values
Feature Encoding	**LabelEncoder**	Converting categorical features into numerical values
Feature Scaling	**StandardScaler**	Standardizing input features before model training
Machine Learning	**scikit-learn**	Building and training classification models
Model Validation	**Cross-Validation**	Evaluating model consistency using 5-fold validation
Hyperparameter Tuning	**GridSearchCV**	Finding suitable parameters for each model
Visualization	**Matplotlib**	Creating class distribution, confusion matrix and accuracy graphs

Table 8: Technology Stack and Its Purpose

7. Code

```
import pandas as pd
import matplotlib.pyplot as plt
import time
from sklearn.model_selection import train_test_split, cross_val_score,
GridSearchCV
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import LinearSVC
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report,
confusion_matrix

df = pd.read_csv("Flight_Delay_Prediction_Dataset_Raw.csv")

print("Dataset loaded successfully")
print("Rows:", df.shape[0], "Columns:", df.shape[1])

# Check unique values

for col in ["Airline", "Weather", "Aircraft_Type", "Holiday",
"Delay_Status"]:
    print(col, ":", df[col].unique())

# Check duplicates and target variable

print("Duplicate rows:", df.duplicated().sum())

print("\nTarget distribution:")
print(df["Delay_Status"].value_counts())
# Check numerical data

df.describe()

# Remove duplicate rows
```

```
duplicates = df.duplicated().sum()
df = df.drop_duplicates().copy()

print("Duplicate rows removed:", duplicates)
print("Rows after removing duplicates:", df.shape[0])

# Impute missing values

num_imputer = SimpleImputer(strategy="median")
df[["Temperature_C", "Wind_Speed_kmph", "Distance_km"]] =
num_imputer.fit_transform(
    df[["Temperature_C", "Wind_Speed_kmph", "Distance_km"]])

cat_imputer = SimpleImputer(strategy="most_frequent")
df[["Airline", "Weather", "Aircraft_Type"]] = cat_imputer.fit_transform(
    df[["Airline", "Weather", "Aircraft_Type"]])

print("Missing values imputed")

# Check missing values after imputation

print(df.isnull().sum())

# Remove outliers

q1 = df["Wind_Speed_kmph"].quantile(0.25)
q3 = df["Wind_Speed_kmph"].quantile(0.75)
iqr = q3 - q1

df = df[
    (df["Wind_Speed_kmph"] >= q1 - 1.5 * iqr) &
    (df["Wind_Speed_kmph"] <= q3 + 1.5 * iqr)
].copy()

print("Outliers removed")
print("Rows after outlier removal:", df.shape[0])

# Encode categorical columns

le = LabelEncoder()
for col in ["Airline", "Origin", "Destination", "Weather",
"Aircraft_Type", "Holiday"]:
```

```
    df[col] = le.fit_transform(df[col])

print("Categorical columns encoded")

# Remove unnecessary columns

df = df.drop(["Flight_ID", "Date"], axis=1)
print("Dataset shape:", df.shape)

# Remove leakage and identifier features

df = df.drop(["Departure_Delay_Min", "Flight_Number"], axis=1)

print("Columns:", df.columns.tolist())
df.head()

# Separate features and target

X = df.drop("Delay_Status", axis=1)
y = df["Delay_Status"]

print("Features shape:", X.shape)
print("Target shape:", y.shape)

# Target Class Distribution

df["Delay_Status"].value_counts().plot(
    kind="pie", autopct="%1.1f%%", startangle=90
)
plt.title("Flight Delay Status Distribution")
plt.ylabel("")
plt.show()

# Check Class Balance

print(df["Delay_Status"].value_counts())
print(df["Delay_Status"].value_counts(normalize=True).mul(100).round(2))

# Check outliers

for col in ["Temperature_C", "Wind_Speed_kmph", "Distance_km"]:
    q1, q3 = df[col].quantile([0.25, 0.75])
```

```
    print(col, "Outliers:", ((df[col] < q1-1.5*(q3-q1)) | (df[col] >
q3+1.5*(q3-q1))).sum())

# Split data into training and testing data

X_train_70, X_test_70, y_train_70, y_test_70 = train_test_split(
    X, y, test_size=0.3, random_state=42
)

X_train_80, X_test_80, y_train_80, y_test_80 = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print("70-30:", X_train_70.shape, X_test_70.shape)
print("80-20:", X_train_80.shape, X_test_80.shape)

# Scale the features

scaler_70 = StandardScaler()
X_train_70 = scaler_70.fit_transform(X_train_70)
X_test_70 = scaler_70.transform(X_test_70)

scaler_80 = StandardScaler()
X_train_80 = scaler_80.fit_transform(X_train_80)
X_test_80 = scaler_80.transform(X_test_80)

print("Feature scaling completed")

# Logistic Regression

lr_70 = LogisticRegression(max_iter=10000)
lr_70.fit(X_train_70, y_train_70)
y_pred_lr_70 = lr_70.predict(X_test_70)

lr_80 = LogisticRegression(max_iter=10000)
lr_80.fit(X_train_80, y_train_80)
y_pred_lr_80 = lr_80.predict(X_test_80)

print("70-30 Accuracy:", accuracy_score(y_test_70, y_pred_lr_70))
print(classification_report(y_test_70, y_pred_lr_70))

print("80-20 Accuracy:", accuracy_score(y_test_80, y_pred_lr_80))
```

```
print(classification_report(y_test_80, y_pred_lr_80))

# KNN

knn_70 = KNeighborsClassifier(n_neighbors=5)
knn_70.fit(X_train_70, y_train_70)
y_pred_knn_70 = knn_70.predict(X_test_70)

knn_80 = KNeighborsClassifier(n_neighbors=5)
knn_80.fit(X_train_80, y_train_80)
y_pred_knn_80 = knn_80.predict(X_test_80)

print("70-30 Accuracy:", accuracy_score(y_test_70, y_pred_knn_70))
print(classification_report(y_test_70, y_pred_knn_70))

print("80-20 Accuracy:", accuracy_score(y_test_80, y_pred_knn_80))
print(classification_report(y_test_80, y_pred_knn_80))

# Decision Tree

dt_70 = DecisionTreeClassifier(random_state=42)
dt_70.fit(X_train_70, y_train_70)
y_pred_dt_70 = dt_70.predict(X_test_70)

dt_80 = DecisionTreeClassifier(random_state=42)
dt_80.fit(X_train_80, y_train_80)
y_pred_dt_80 = dt_80.predict(X_test_80)

print("70-30 Accuracy:", accuracy_score(y_test_70, y_pred_dt_70))
print(classification_report(y_test_70, y_pred_dt_70))

print("80-20 Accuracy:", accuracy_score(y_test_80, y_pred_dt_80))
print(classification_report(y_test_80, y_pred_dt_80))

# Naive Bayes

nb_70 = GaussianNB()
nb_70.fit(X_train_70, y_train_70)
y_pred_nb_70 = nb_70.predict(X_test_70)

nb_80 = GaussianNB()
nb_80.fit(X_train_80, y_train_80)
```

```
y_pred_nb_80 = nb_80.predict(X_test_80)

print("70-30 Accuracy:", accuracy_score(y_test_70, y_pred_nb_70))
print(classification_report(y_test_70, y_pred_nb_70))

print("80-20 Accuracy:", accuracy_score(y_test_80, y_pred_nb_80))
print(classification_report(y_test_80, y_pred_nb_80))

# Random Forest

rf_70 = RandomForestClassifier(n_estimators=100, random_state=42)
rf_70.fit(X_train_70, y_train_70)
y_pred_rf_70 = rf_70.predict(X_test_70)

rf_80 = RandomForestClassifier(n_estimators=100, random_state=42)
rf_80.fit(X_train_80, y_train_80)
y_pred_rf_80 = rf_80.predict(X_test_80)

print("70-30 Accuracy:", accuracy_score(y_test_70, y_pred_rf_70))
print(classification_report(y_test_70, y_pred_rf_70))

print("80-20 Accuracy:", accuracy_score(y_test_80, y_pred_rf_80))
print(classification_report(y_test_80, y_pred_rf_80))

# Linear SVM

svm_80 = LinearSVC(
    C=1.0,
    class_weight="balanced",
    random_state=42,
    max_iter=2000
)

svm_80.fit(X_train_80, y_train_80)
y_pred_svm_80 = svm_80.predict(X_test_80)

print("SVM Accuracy:", round(accuracy_score(y_test_80, y_pred_svm_80),
4))
print(classification_report(y_test_80, y_pred_svm_80))

# Confusion Matrix Visualization
```

```
cm = confusion_matrix(y_test_80, y_pred_rf_80)

plt.figure(figsize=(7, 5))

plt.imshow(cm, interpolation="nearest")

plt.title("Confusion Matrix - Random Forest (80-20)")
plt.xlabel("Predicted Label")
plt.ylabel("Actual Label")

plt.xticks([0, 1], ["No Delay", "Delay"])
plt.yticks([0, 1], ["No Delay", "Delay"])

# Display values inside the matrix

for i in range(2):
    for j in range(2):
        plt.text(j, i, cm[i, j],
                 ha="center",
                 va="center")

plt.colorbar()

plt.tight_layout()
plt.show()

# Compare model accuracies

results = {
    "Logistic Regression": [
        accuracy_score(y_test_70, y_pred_lr_70),
        accuracy_score(y_test_80, y_pred_lr_80)    ],
    "KNN": [
        accuracy_score(y_test_70, y_pred_knn_70),
        accuracy_score(y_test_80, y_pred_knn_80)    ],
    "Decision Tree": [
        accuracy_score(y_test_70, y_pred_dt_70),
        accuracy_score(y_test_80, y_pred_dt_80)    ],
    "Naive Bayes": [
        accuracy_score(y_test_70, y_pred_nb_70),
        accuracy_score(y_test_80, y_pred_nb_80)    ],
    "Random Forest": [
```

```
        accuracy_score(y_test_70, y_pred_rf_70),
        accuracy_score(y_test_80, y_pred_rf_80)    ],
   "SVM": [
    None,
    accuracy_score(y_test_80, y_pred_svm_80)
]}

comparison = pd.DataFrame(results, index=["70-30", "80-20"]).T
comparison.columns = ["70-30 Accuracy", "80-20 Accuracy"]

print(comparison)

# Model Training Time

models = {
    "Logistic Regression": LogisticRegression(max_iter=10000),
    "KNN": KNeighborsClassifier(n_neighbors=5),
    "Decision Tree": DecisionTreeClassifier(random_state=42),
    "Naive Bayes": GaussianNB(),
    "Random Forest": RandomForestClassifier(n_estimators=100,
random_state=42),
    "SVM": LinearSVC(class_weight="balanced", max_iter=2000,
random_state=42)
}

for name, model in models.items():
    start = time.time()
    model.fit(X_train_80, y_train_80)
    print(name, "Time:", round(time.time() - start, 3), "sec")

# 5-Fold Cross Validation

models = {
    "Logistic Regression": LogisticRegression(max_iter=10000),
    "KNN": KNeighborsClassifier(n_neighbors=5),
    "Decision Tree": DecisionTreeClassifier(random_state=42),
    "Naive Bayes": GaussianNB(),
    "Random Forest": RandomForestClassifier(n_estimators=100,
random_state=42),
    "SVM": LinearSVC(
        C=1.0,
        class_weight="balanced",
```

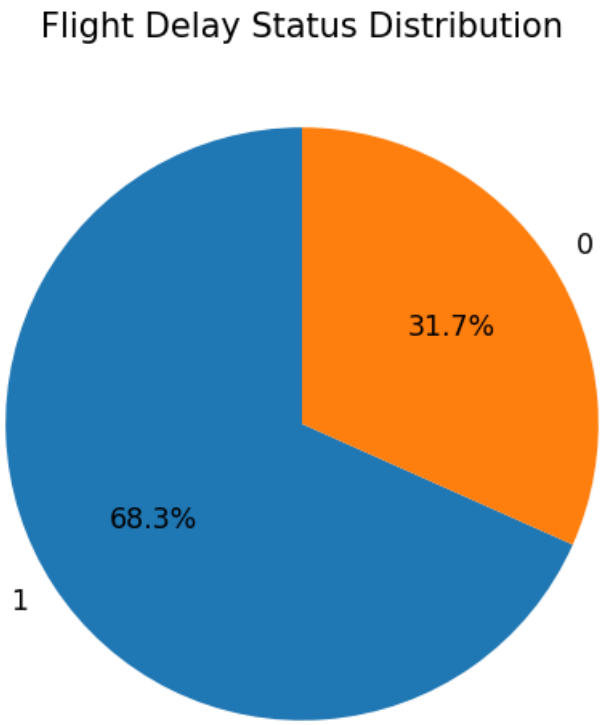
```
        max_iter=2000,
        random_state=42
    )
}

for name, model in models.items():
    scores = cross_val_score(model, X_train_80, y_train_80, cv=5)
    print(name, "Mean:", round(scores.mean(), 4),
          "Std:", round(scores.std(), 4))

# Grid Search for all models

param_grids = {
    "Logistic Regression": (
        LogisticRegression(max_iter=10000),
        {"C": [0.1, 1, 10]}
    ),

    "KNN": (
        KNeighborsClassifier(),
        {"n_neighbors": [3, 5, 7]}
    ),

    "Decision Tree": (
        DecisionTreeClassifier(random_state=42),
        {"max_depth": [10, 20, None]}
    ),

    "Naive Bayes": (
        GaussianNB(),
        {"var_smoothing": [1e-9, 1e-8, 1e-7]}
    ),

    "Random Forest": (
        RandomForestClassifier(random_state=42),
        {"n_estimators": [50, 100], "max_depth": [10, 20]}
    ),

    "SVM": (
        LinearSVC(class_weight="balanced", random_state=42),
        {"C": [0.1, 1, 10]}
    )
```

```
}

best_models = {}

for name, (model, params) in param_grids.items():

    grid = GridSearchCV(
        model,
        params,
        cv=5,
        scoring="accuracy",
        n_jobs=1
    )

    grid.fit(X_train_80, y_train_80)

    best_models[name] = grid.best_estimator_

    print(name)
    print("Best Parameters:", grid.best_params_)
    print("Best CV Accuracy:", round(grid.best_score_, 4))
    print()

# Tuned Model Results

tuned_results = {}
for name, model in best_models.items():
      y_pred = model.predict(X_test_80)
      accuracy = accuracy_score(y_test_80, y_pred)
      tuned_results[name] = accuracy
      print(name, "Accuracy:", round(accuracy, 4))

# Tuned Model Accuracy Comparison

models_name = list(tuned_results.keys())
accuracies = list(tuned_results.values())

plt.figure(figsize=(10, 5))
plt.bar(models_name, accuracies)

plt.xlabel("Machine Learning Algorithm")
plt.ylabel("Accuracy")
```

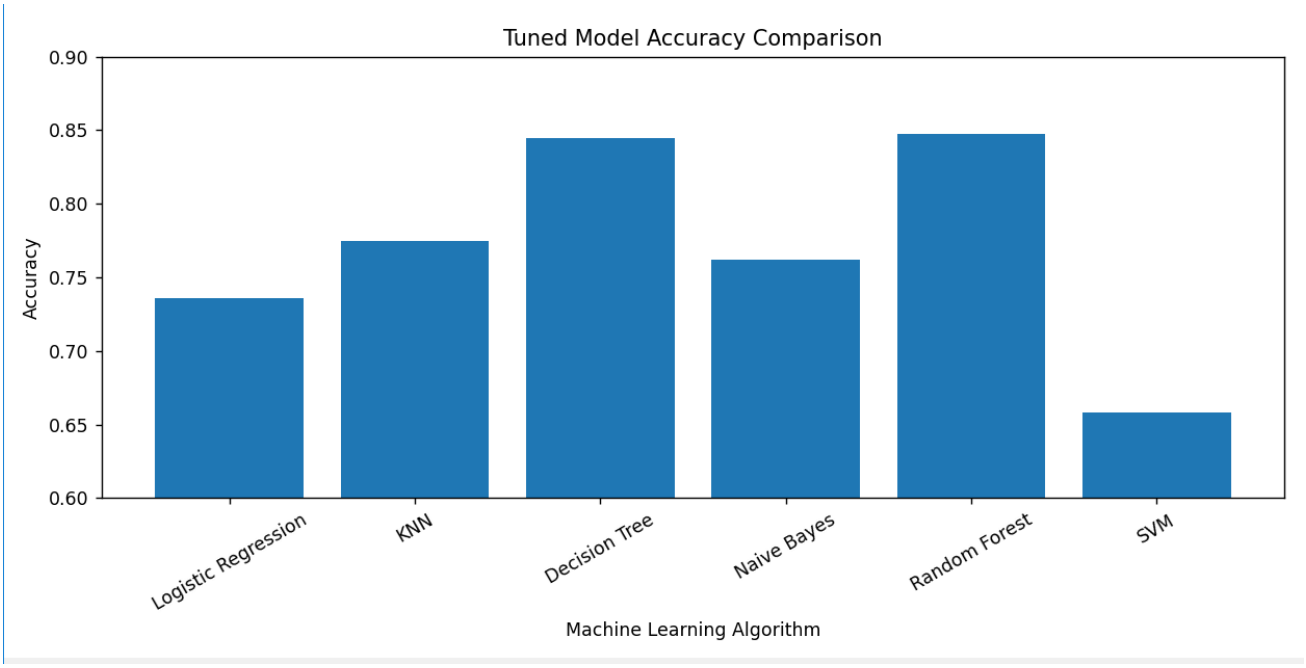
```
plt.title("Tuned Model Accuracy Comparison")

plt.xticks(rotation=30)
plt.ylim(0.60, 0.90)

plt.tight_layout()
plt.show()

# Confusion Matrix for the Best Model

best_name = max(tuned_results, key=tuned_results.get)
best_model = best_models[best_name]
y_pred_best = best_model.predict(X_test_80)

cm = confusion_matrix(y_test_80, y_pred_best)

print("Best Model:", best_name)
print("Confusion Matrix:")
print(cm)

# Classification Report for the Best Model

print("Best Model:", best_name)
print(classification_report(y_test_80, y_pred_best))

# Final Model Comparison

final_results = pd.DataFrame({
    "Model": list(tuned_results.keys()),
    "Accuracy": list(tuned_results.values())
}).sort_values("Accuracy", ascending=False)

print(final_results)

# Best Model Result

print("Best Model:", best_name)
print("Train-Test Split: 80-20")
print("Accuracy:", round(tuned_results[best_name], 4))

# Final Model Comparison
```

```
final_results = pd.DataFrame({
    "Model": list(tuned_results.keys()),
    "Accuracy": list(tuned_results.values())
}).sort_values("Accuracy", ascending=False)

print(final_results)

# Final Best Model

print("Best Model:", best_name)
print("Accuracy:", round(tuned_results[best_name], 4))

# Final Model Accuracy Graph

plt.figure(figsize=(10, 5))

plt.bar(final_results["Model"], final_results["Accuracy"])

plt.xlabel("Model")
plt.ylabel("Accuracy")
plt.title("Final Model Accuracy Comparison")

plt.xticks(rotation=30)
plt.ylim(0.60, 0.90)

plt.tight_layout()
plt.show()
```

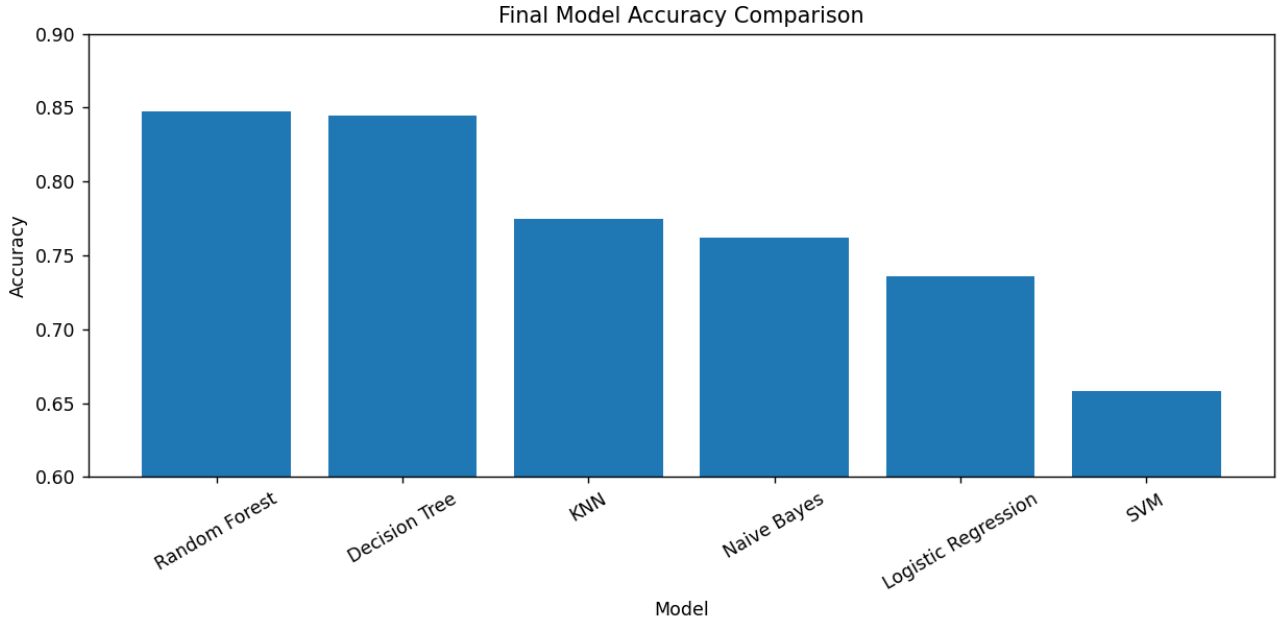
8. Visualization



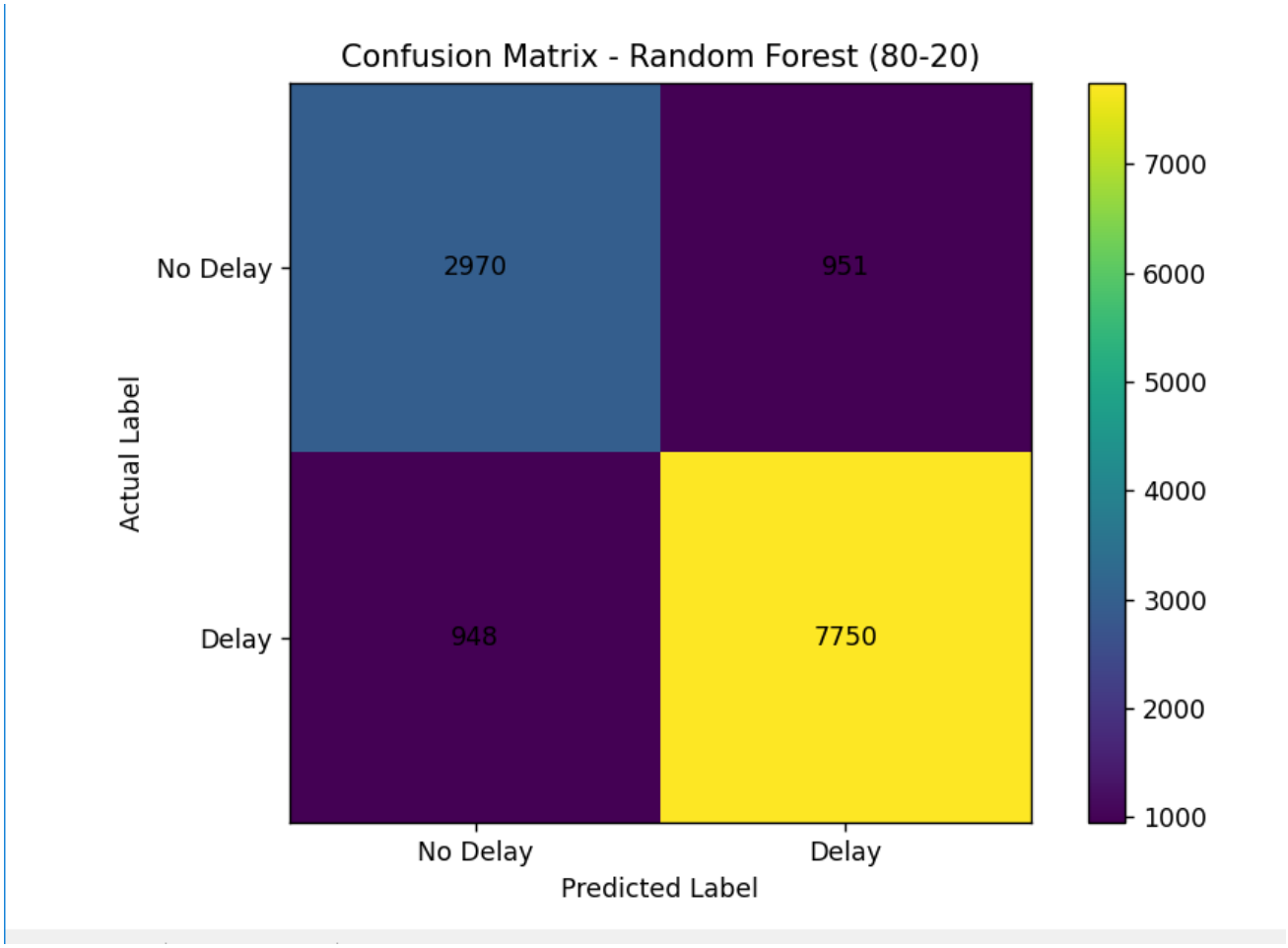
8.1 Target Class Distribution



8.2 5-Fold Cross-Validation Accuracy



8.3 Final Model Accuracy Comparison



8.4 Confusion Matrix of Best Model

9. Conclusion

In this project, a Machine Learning-based approach was developed to predict whether a flight would be delayed or not. The dataset was cleaned and prepared by handling missing values, removing duplicate records and outliers, encoding categorical features, removing unnecessary and leakage-related features, and applying feature scaling.

Six classification algorithms were implemented and compared: Logistic Regression, K-Nearest Neighbors (KNN), Decision Tree, Naive Bayes, Random Forest, and Linear Support Vector Machine (SVM). The models were evaluated using accuracy, precision, recall, F1-score, confusion matrix, and 5-Fold Cross-Validation. GridSearchCV was also used to improve the performance by finding suitable hyperparameters.

The main findings of the project are:

- **Random Forest achieved the best overall performance** among the tested models.
- The final tuned Random Forest model achieved **84.76% test accuracy** and **84.82% cross-validation accuracy** using the 80–20 split.
- For the **Delay** class, the model achieved **0.89 precision, 0.89 recall, and 0.89 F1-score**.
- The results show that Machine Learning can effectively identify flight delay patterns using flight-related and environmental features.

Overall, the project successfully demonstrates the application of Machine Learning for flight delay prediction. Random Forest was selected as the final model because it provided the best performance on the given dataset. In the future, the system can be improved by using real-time weather, airport traffic, historical flight data, and larger datasets to achieve better prediction performance.

Thank you...